MYONLINEJET

INSTALLATION AND ENVIRONMENT GUIDE

For local development, test deployment, and production handover

Field	Value
Project	MyOnlineJet full-stack web platform
Repository	https://github.com/Abhiman67/MOJ-LIVE
Test website	https://test.myonlinejet.com/
Receiving owner	[Boss name and company email]
Prepared by	[Your name]
Guide date	20 October 2026

This guide contains variable names and placeholders only. Never place actual passwords, database URLs, OAuth secrets, SMTP passwords, AWS keys, or private keys in this document or in Git.

1. Application overview

MyOnlineJet is a Next.js 15 application with a React frontend, Node.js route handlers, MongoDB persistence through Mongoose, Auth.js authentication, SMTP notifications, protected user/profile routes, and an administrator dashboard. The project is located in the frontend directory of the repository.

2. Required software

Requirement	Recommended setup
Operating system	macOS, Linux, or Windows with a terminal
Node.js	Node.js 20 or newer
Package manager	npm included with Node.js
Database	MongoDB local instance or MongoDB Atlas
Version control	Git
Production process	PM2 on a Node.js server
Reverse proxy	Nginx, Apache, or hosting equivalent with HTTPS

3. Get the source code

Open a terminal and run:

```
git clone https://github.com/Abhiman67/MOJ-LIVE.git cd MOJ-LIVE/frontend
```

If the repository is already present, use git status and git pull only after confirming the correct branch and preserving any local work.

4. Install dependencies

Run the following from the frontend directory:

```
npm ci
```

npm ci uses package-lock.json and installs the recorded dependency versions. Use npm install only when intentionally changing dependencies.

5. Create the environment file

Copy the template and create a local environment file:

```
cp .env.example .env
```

Open .env in a local editor and replace placeholders with safe development values. The .env file is private and must never be committed.

6. Environment variable reference

Variable	What to enter
MONGODB_URI	Local MongoDB URI or the company-approved Atlas URI. Use a restricted database user.
AUTH_SECRET	Long random Auth.js secret. Generate a separate value for local, test, and production.
AUTH_URL	Local: http://localhost:3000. Test/production: the exact HTTPS application origin.
AUTH_CREDENTIALS_ENABLED	true only when email/password login is intentionally enabled.
NEXT_PUBLIC_AUTH_CREDENTIALS	Keep aligned with the server credentials switch.
Google OAuth values	AUTH_GOOGLE_ID and AUTH_GOOGLE_SECRET, if enabled.
Apple OAuth values	AUTH_APPLE_ID and AUTH_APPLE_SECRET, if enabled.
LinkedIn OAuth values	AUTH_LINKEDIN_ID and AUTH_LINKEDIN_SECRET, if enabled.
NEXT_PUBLIC_AUTH flags	true only for providers with configured credentials.
SMTP_HOST / SMTP_PORT	SMTP provider host and port.
SMTP_USER / SMTP_PASS	SMTP account or app-password values.
FROM_EMAIL	Verified sender address.
NOTIFICATION_EMAIL	Company recipient for form notifications.
ADMIN_EMAIL / ADMIN_PASSWORD	Seed-only values; use a unique password of at least 12 characters.
ALLOW_SEED	false or unset normally; true only for a controlled seed operation.

Public NEXT_PUBLIC_* values are embedded into the browser bundle. Never place secrets in a NEXT_PUBLIC_* variable.

7. Local database setup

For local MongoDB, create a database named myonlinejet or use the company-approved development database. Confirm the MongoDB service is running, then set MONGODB_URI in .env. For Atlas, confirm the database user, password, network access, cluster, and database name with the company owner.

Optional controlled admin seed:

```
ALLOW_SEED=true node scripts/seed.mjs # Disable or remove ALLOW_SEED immediately after the operation
```

Never run the seed script against production without explicit approval and a backup.

8. Start local development

Run:

```
npm run dev
```

Open http://localhost:3000. Check the homepage, /auth, /profile, /dashboard, and the public pages. If the application opens but authentication fails, check MONGODB_URI, AUTH_SECRET, AUTH_URL, and the browser origin.

9. Local verification

Check	Expected result
Homepage	Website renders without missing assets or console errors.
Navigation	Main and mobile navigation links resolve correctly.
Authentication	Configured login method works and logout ends the session.
Profile	Authenticated user can load and save profile data.
Forms	Test submissions are stored in MongoDB.
Dashboard	User sees permitted submissions; admin sees authorized admin features.
Email	SMTP test delivers to the controlled test recipient.
Build	Production build completes successfully.

Verification commands:

```
npm run lint npx tsc --noEmit npm run build
```

10. OAuth setup

For each enabled provider, create or use a company-owned OAuth application and add the correct callback URL for the environment. The usual Auth.js callback pattern is /api/auth/callback/. Use the exact HTTPS domain in test or production. Do not use localhost credentials in production.

11. SMTP setup

Configure SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, FROM_EMAIL, and NOTIFICATION_EMAIL. Confirm the sender is verified by the email provider. Run the project email test only with a controlled recipient and never print SMTP passwords in logs.

12. Production installation

Confirm the server, domain, DNS, TLS, MongoDB, SMTP, and company-owned credentials before starting. The documented deployment model uses Node.js, PM2, and a reverse proxy forwarding HTTPS traffic to port 3000.

On the server:

```
git clone https://github.com/Abhiman67/MOJ-LIVE.git cd MOJ-LIVE/frontend npm ci # Configure the production environment outside Git npm run lint npx tsc --noEmit npm run build pm2 start ecosystem.config.js pm2 save
```

The commands above assume pm2 is installed. The repository also contains deploy.sh for the documented routine deployment process. Review the checked-out commit before running automated deployment.

13. Reverse proxy and domain

• Terminate TLS at the reverse proxy.

• Forward requests to the application on port 3000.

• Forward the original host and protocol headers.

• Use the public HTTPS origin in AUTH_URL.

• Configure OAuth callback URLs against the public origin.

• Verify DNS points to the correct hosting server.

14. Production smoke test

Test	Pass condition
Public site	Homepage and major routes load over HTTPS.
Auth	Configured provider login and logout work.
Protected route	Unauthenticated access redirects to /auth.
Database	A controlled test submission is persisted.
Email	A controlled notification is delivered.
Admin	Admin can review a test submission and update status.
Operations	PM2 status is healthy and logs show no startup error.
SEO	/sitemap.xml and /robots.txt respond correctly.

15. Troubleshooting

Problem	Check
MONGODB_URI is not defined	Confirm .env exists in the frontend directory and the process has loaded it.
Authentication fails	Check AUTH_SECRET, AUTH_URL, MongoDB connectivity, callback URL, and provider flags.
OAuth callback error	Use the exact deployed HTTPS origin and /api/auth/callback/<provider>.
Email not received	Check SMTP values, verified sender, recipient, provider logs, and spam folder.
Form returns unauthorized	Sign in first and confirm the session cookie and AUTH_SECRET are consistent.
Admin access denied	Confirm the signed-in user has the admin role in MongoDB.
Build uses old public flags	Restart after changing NEXT_PUBLIC_* values and rebuild production.
Database connection fails	Check Atlas IP allowlist, database user permissions, URI encoding, and server network access.
PM2 process unhealthy	Check pm2 logs, environment loading, port 3000, reverse proxy, and the latest build.

16. Security and credential handling

• Keep .env and production secret values outside Git.

• Use a company password manager or secret manager.

• Never send secrets in chat, normal email, or screenshots.

• Use separate local, test, and production credentials.

• Enable two-factor authentication for GitHub, hosting, domain, database, email, OAuth, AWS, analytics, and support services.

• Rotate credentials after the handover if they were shared informally.

• Remove personal access only after the receiving owner has verified independent access.

17. Installation handover checklist

Item	Status
Repository cloned by receiving owner	[ ]
Dependencies installed with npm ci	[ ]
Local .env configured safely	[ ]
MongoDB connection verified	[ ]
Authentication verified	[ ]
SMTP delivery verified	[ ]
Production environment configured	[ ]
Production build completed	[ ]
PM2 and reverse proxy verified	[ ]
Rollback process demonstrated	[ ]
Credentials stored in company vault	[ ]

Item	Status
Original personal access removed	[ ]

18. Support references

Use the following repository documents alongside this guide: docs/DEVELOPMENT.md, docs/ENVIRONMENT.md, docs/DEPLOYMENT.md, docs/API.md, docs/ARCHITECTURE.md, docs/OPERATIONS.md, docs/security-and-testing.md, docs/HANDOVER.md, docs/CREDENTIALS-INVENTORY.md, docs/HANDOVER-TESTING.md, docs/KNOWN-ISSUES.md, and docs/HANDOVER-SIGNOFF.md.

Receiver acknowledgement: I have received this installation guide, understand the environment setup and deployment process, and will store actual secrets only in the company-approved secure system.

Receiving owner: _____ Signature: _____ Date: _____